

# **VITYARTHI PROJECT**

## **OPEN SOURCE AUDIT PROJECT**

Course: Open Source Software

- ❖ Student Name: [ABHISHEK A]
- ❖ Registration Number: [24MEI10009]
- ❖ Slot: [A24]
- ❖ Date of Submission: [29-03-26]

**Chosen Software: Python**

---

### **TABLE OF CONTENTS**

- Introduction
- Part A1 — Origin of Python
- Part A2 — License
- Part A3 — Ethics
- Part B — Linux Footprint
- Part C — FOSS Ecosystem
- Part D — Open Source vs Proprietary
- Conclusion
- Scripts
- References

### **INTRODUCTION**

In today's digital world, a large portion of the software we use every day is built using open-source technologies. Open-source software is developed in a way that allows anyone to view, modify, and share the source code. I understand

that, This approach encourages collaboration, transparency, and continuous improvement.

This project focuses on understanding the concept of open source by studying a real example. For this purpose, Python has been chosen as the subject of this audit. Python is one of the most widely used programming languages in the world and is known for its simplicity and readability. People find it very easy to use.

The aim of this project is not only to understand what Python does, but also to explore why it was created, how it is licensed, and how it fits into the larger open-source ecosystem. In addition, the project includes practical work using Linux and shell scripting to demonstrate how open-source tools are used in real systems.

Through this project, I have learned how to connect both theoretical knowledge and practical skills. It also helped me understand the importance of open-source software in modern computing and how it continues to shape the technology we use today.

## **Part A1 — Origin Of Python**

Python was created by Guido van Rossum in the late 1980s and released in 1991. At that time, programming languages like C and Java were very powerful but difficult to learn and use. But many programmers found them too complex for simple tasks.

Guido wanted to create a language that was easy to read, simple to use, and powerful at the same time. His goal was to make programming more accessible to everyone, not just experts. He believed that code should be written in a way that is almost like reading English that easy.

Python solved the problem of complexity in programming. It allowed developers to write fewer lines of code and still achieve more functionality. Because of this, it quickly became popular among beginners as well as professionals.

Instead of keeping it private, Guido released Python as open-source software. This allowed developers from all over the world to contribute, improve it, and make it what it is today.

## **Part A2 — License**

Python uses the PSF (Python Software Foundation) License. This license allows users to freely use and distribute the software.

The four freedoms of free software include:

- Freedom to run the program
- Freedom to study the code
- Freedom to modify it
- Freedom to share it

Python provides all these freedoms.

As we look, a company can modify Python and even use it in their own product, but they must follow the license terms. Unlike GPL, Python's license is more flexible and does not force companies to release their changes.

The difference between “free as in free beer” and “free as in freedom” is important and Python is free in terms of freedom, meaning users have control over the software, not just free access.

## **Part A3 — Ethics**

Open source software promotes sharing of knowledge and collaboration. It allows people from different parts of the world to work together and improve software.

As it all, not all software should necessarily be open source. Some companies invest large amounts of money and may need to protect their work.

It can be debated whether it is ethical for companies to earn profit using open-source software. On one hand, they benefit from community work. And on the other hand, they also contribute improvements and innovations.

The idea of “standing on the shoulders of giants” means building on existing knowledge. Open source supports this idea and encourages faster innovation.

## **Part B — Linux Footprint**

Python is usually already installed in most Linux systems. If it is not available, it can be easily installed using package managers like `apt` or `yum`, depending on the Linux distribution being used.

The Python executable is generally found in directories like `/usr/bin/python3`. Unlike many other software, Python does not need much configuration to work. Most of its libraries and modules are stored in folders such as `/usr/lib/python3`, where the system keeps all the required files.

Python runs under the currently logged-in user instead of a separate system user. This is important from a security point of view because it ensures that Python programs can only access through the files and data that the user has permission to use.

Also, Python does not run continuously in the background like services such as web servers. It only runs when we execute a program using commands like `python3 filename.py`. Once the program finishes, it stops automatically as it is.

Updates for Python are handled through the Linux package manager. The open-source community regularly provides updates and security fixes, which can also be installed easily using system update commands.

## **Part C — FOSS Ecosystem**

Python does not work in its own way; it is part of a very large open-source ecosystem. It depends on different system libraries and tools like GCC to run properly and efficiently on a Linux system.

Over time, Python has inspired the development of many useful tools and frameworks. For example, Django is used for web development, NumPy is used for scientific calculations, and TensorFlow is used in machine learning. These tools have made Python very powerful and popular in different fields.

Even though Python is not directly a part of the LAMP stack, it can still be used in web development instead of PHP. In fact, many modern websites and applications use Python because it is easy to learn and flexible so that people can use it easily.

Python also has a very large and active community. Developers from all over the world contribute to it by sharing code, fixing bugs, and improving features. Platforms like GitHub, online forums, and conferences help in this collaboration. The Python Software Foundation also plays an important role in managing and guiding the future development of Python.

## **Part D — Open Source vs Proprietary**

| <b>Feature</b> | <b>Python (Open Source)</b> | <b>Proprietary Alternative<br/>(e.g., MATLAB)</b> |
|----------------|-----------------------------|---------------------------------------------------|
| Cost           | Free                        | Paid (expensive license)                          |
| Security       | Code is open for review     | Closed source                                     |
| Support        | Community support           | Official paid support                             |
| Freedom        | Can modify and share        | Limited modification                              |
| Control        | Community driven            | Company controlled                                |

## **Conclusion**

In conclusion, this project helped me gain a deeper understanding of open-source software and its importance in today's technological world. By studying Python, I was able to see how a simple idea, making programming easier and more accessible, grew into one of the most widely used tools globally. It clearly shows how collaboration and sharing knowledge can lead to powerful and long-lasting innovations.

One of the most interesting things I learned is that open-source software is not just about free access, but about freedom and flexibility. Users are not restricted; they can study the code, modify it, and even build new tools on top of it. This creates a learning environment where anyone can contribute, regardless of their background.

Working on the Linux environment and writing shell scripts also gave me practical exposure. It helped me understand how software actually runs in a system, how automation works, and how simple commands can perform useful tasks. This practical experience made the concepts much clearer compared to just reading theory.

At the same time, I do understand that open-source software has its challenges. It depends heavily on community contributions, and sometimes it may lack dedicated support compared to proprietary software. However, the advantages such as transparency, cost-effectiveness, and innovation outweigh these limitations.

Overall, I will surely recommend Python for both learning and real-world applications because of its simplicity, versatility, and strong community support. This project has also made me more interested in exploring open-source projects in the future and possibly contributing to them in my own way. This project improved my understanding of Linux and open-source concepts.

## **Script 1**

```
#!/bin/bash
```

```
echo "==== System Identity ====="
```

```
echo "User: $(whoami)"
```

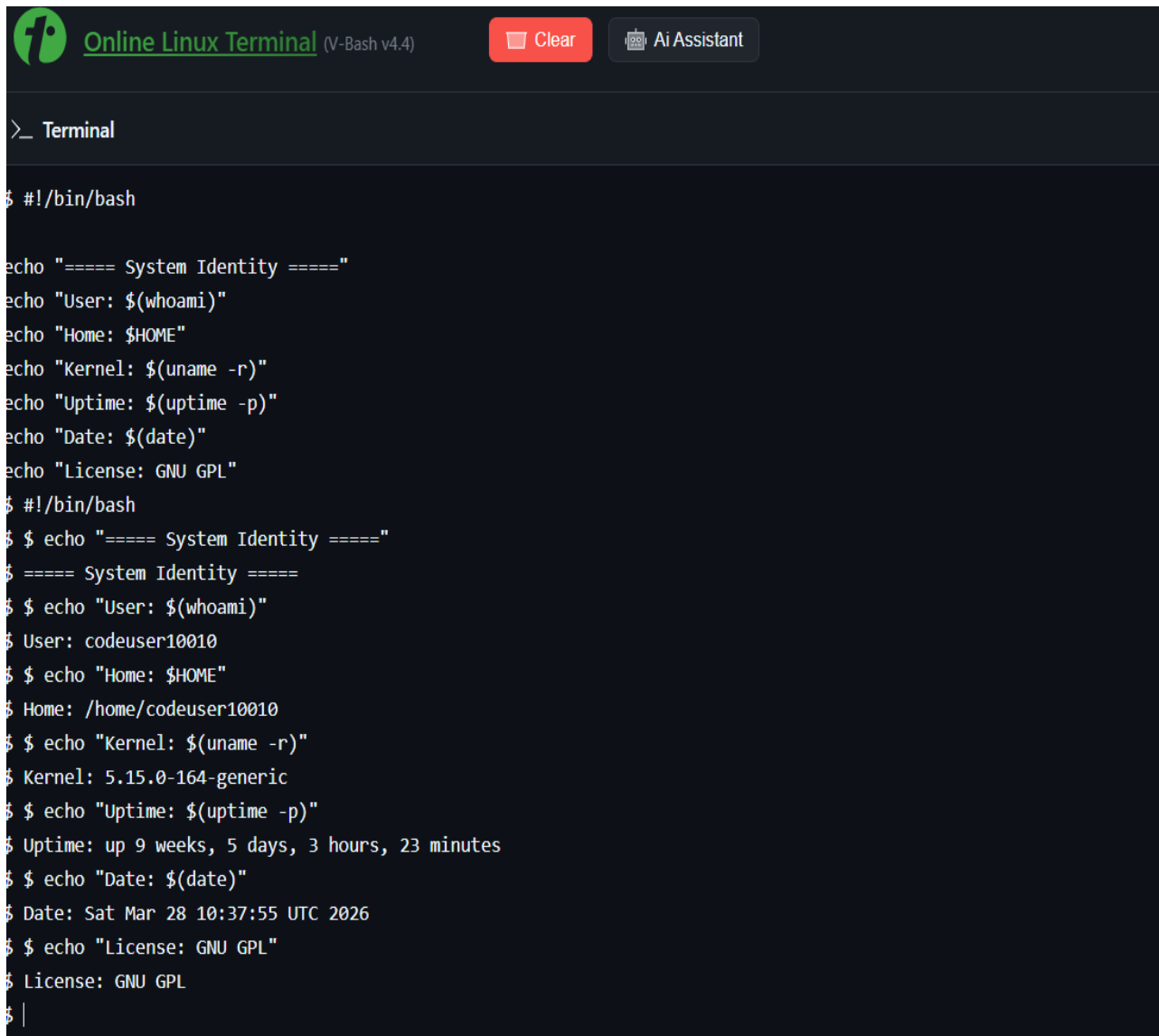
```
echo "Home: $HOME"
```

```
echo "Kernel: $(uname -r)"
```

```
echo "Uptime: $(uptime -p)"
```

```
echo "Date: $(date)"
```

```
echo "License: GNU GPL"
```



The screenshot shows a web-based terminal interface titled "Online Linux Terminal (V-Bash v4.4)". It features a dark theme and a header bar with a logo, a "Clear" button, and an "Ai Assistant" button. The terminal window displays the following commands and their outputs:

```
>_ Terminal

$ #!/bin/bash

echo "==== System Identity ====="
echo "User: $(whoami)"
echo "Home: $HOME"
echo "Kernel: $(uname -r)"
echo "Uptime: $(uptime -p)"
echo "Date: $(date)"
echo "License: GNU GPL"
$ #!/bin/bash
$ $ echo "==== System Identity ====="
$ ===== System Identity =====
$ $ echo "User: $(whoami)"
$ User: codeuser10010
$ $ echo "Home: $HOME"
$ Home: /home/codeuser10010
$ $ echo "Kernel: $(uname -r)"
$ Kernel: 5.15.0-164-generic
$ $ echo "Uptime: $(uptime -p)"
$ Uptime: up 9 weeks, 5 days, 3 hours, 23 minutes
$ $ echo "Date: $(date)"
$ Date: Sat Mar 28 10:37:55 UTC 2026
$ $ echo "License: GNU GPL"
$ License: GNU GPL
$ |
```

## Script 2

```
#!/bin/bash
```

```
PACKAGE="python3"
```

```
if dpkg -l | grep -q $PACKAGE
```

```
then
```

```
    echo "$PACKAGE is installed"
```

```
    python3 --version
```

```
else
```

```
    echo "$PACKAGE is not installed"
```

```
fi
```

```
case $PACKAGE in
```

```
    python3) echo "Python: easy and powerful programming language" ;;
```

```
    *) echo "Unknown software" ;;
```

```
esac
```

```
>_ Terminal

$ #!/bin/bash

PACKAGE="python3"

if dpkg -l | grep -q $PACKAGE
then
    echo "$PACKAGE is installed"
    python3 --version
else
    echo "$PACKAGE is not installed"
fi

case $PACKAGE in
    python3) echo "Python: easy and powerful programming language" ;;
    *) echo "Unknown software" ;;
esac

$ #!/bin/bash
$ $ PACKAGE="python3"
$ $ if dpkg -l | grep -q $PACKAGE
$ >
$ > then
$ >
$ >     echo "$PACKAGE is installed"
$ >
$ >     python3 --version
```



```
$ >
$ > else
$ >
$ >     echo "$PACKAGE is not installed"
$ >
$ > fi
$ python3 is installed
$ Python 3.12.3
$ $ case $PACKAGE in
$ >
$ >     python3) echo "Python: easy and powerful programming language" ;;
$ >
$ >     *) echo "Unknown software" ;;
$ >
$ > esac
$ Python: easy and powerful programming language
$ |
```

### **Script 3**

```
#!/bin/bash
DIRS=("/etc" "/home" "/usr/bin")
for DIR in "${DIRS[@]}"
do
    echo "Directory: $DIR"
    ls -ld $DIR
    du -sh $DIR
    echo "-----"
done
```



>\_ Terminal

```
$ #!/bin/bash

DIRS=("/etc" "/home" "/usr/bin")

for DIR in "${DIRS[@]}"
do
    echo "Directory: $DIR"
    ls -ld $DIR
    du -sh $DIR
    echo "-----"
done
$ #!/bin/bash
$ $ DIRS=("/etc" "/home" "/usr/bin")
$ $ for DIR in "${DIRS[@]}"
$ >
$ > do
$ >
$ >     echo "Directory: $DIR"
$ >
$ >     ls -ld $DIR
$ >
$ >     du -sh $DIR
$ >
$ >     echo "-----"
$ >
```

```
$ > echo "-----"
$ >
$ > done
$ Directory: /etc
$ drwxr-xr-x 1 root root 4096 Aug 28 2025 /etc
$ du: cannot read directory '/etc/credstore': Permission denied
$ du: cannot read directory '/etc/credstore.encrypted': Permission denied
$ du: cannot read directory '/etc/ssl/private': Permission denied
$ 6.3M /etc
$ -----
$ Directory: /home
$ drwxr-xr-x 1 root root 4096 Dec 17 2024 /home
$ du: cannot read directory '/home/ubuntu': Permission denied
$ du: cannot read directory '/home/cg/root/69c300784aed9': Permission denied
$ du: cannot read directory '/home/cg/root/69b1539ab848a': Permission denied
$ du: cannot read directory '/home/cg/root/69c3051079d95': Permission denied
$ du: cannot read directory '/home/cg/root/69c7ae6dd5691': Permission denied
$ du: cannot read directory '/home/cg/root/69c300aa72c60': Permission denied
$ du: cannot read directory '/home/cg/root/69c3007206327': Permission denied
$ du: cannot read directory '/home/cg/root/69c2f50906f89': Permission denied
$ du: cannot read directory '/home/cg/root/69c2f5f7a9639': Permission denied
$ du: cannot read directory '/home/cg/root/69c3e22f9e50c': Permission denied
$ du: cannot read directory '/home/cg/root/5601bbdd': Permission denied
$ du: cannot read directory '/home/cg/root/95f3b3fd': Permission denied
$ du: cannot read directory '/home/webmaster': Permission denied
$ du: cannot read directory '/home/rexx': Permission denied
```

```
$ du: cannot read directory '/home/objc': Permission denied
$ 13M /home
$ -----
$ Directory: /usr/bin
$ drwxr-xr-x 1 root root 12288 Jun 9 2025 /usr/bin
$ 812M /usr/bin
$ -----
$
```

## **Script 4**

```
#!/bin/bash
LOGFILE=$1
COUNT=0
if [ ! -f "$LOGFILE" ]
then
    echo "File not found"
    exit 1
fi
while read LINE
do
    if echo "$LINE" | grep -i "error" > /dev/null
    then
        COUNT=$((COUNT+1))
    fi
done < $LOGFILE
echo "Total errors: $COUNT"
```



>\_ Terminal

```
$ #!/bin/bash

LOGFILE=$1
COUNT=0

if [ ! -f "$LOGFILE" ]
then
    echo "File not found"
    exit 1
fi

while read LINE
do
    if echo "$LINE" | grep -i "error" > /dev/null
    then
        COUNT=$((COUNT+1))
    fi
done < $LOGFILE

echo "Total errors: $COUNT"
$ #!/bin/bash
$ $ LOGFILE=$1
$ $ COUNT=0
$ $ if [ ! -f "$LOGFILE" ]
$ >
```

```
$ >
$ > then
$ >
$ >     echo "File not found"
$ >
$ >     exit 1
$ >
$ > fi
$ File not found
$ exit
$
$
```

## **Script 5**

```
#!/bin/bash
```

```
read -p "Tool: " TOOL
```

```
read -p "Freedom means: " FREEDOM
```

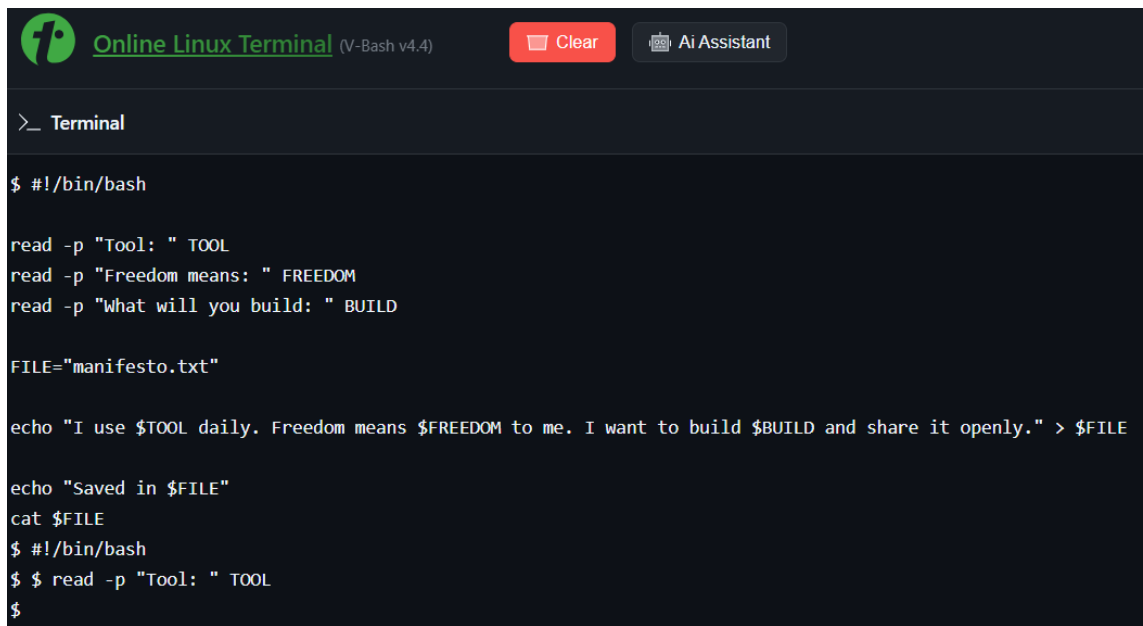
```
read -p "What will you build: " BUILD
```

```
FILE="manifesto.txt"
```

```
echo "I use $TOOL daily. Freedom means $FREEDOM to me. I want  
to build $BUILD and share it openly." > $FILE
```

```
echo "Saved in $FILE"
```

```
cat $FILE
```

A screenshot of an 'Online Linux Terminal' window. The window has a dark theme and a header bar with a logo, the title 'Online Linux Terminal (V-Bash v4.4)', and buttons for 'Clear' and 'Ai Assistant'. The terminal content shows the script being executed line by line. The first part of the script (#!/bin/bash, read commands, FILE assignment, echo command, and cat command) is shown in white text. The second part of the script (#!/bin/bash, read command, and a new prompt) is shown in a lighter gray text, indicating it was not yet executed. The prompt character is '\$'.

## **REFERENCES**

1. Python Official Documentation — <https://docs.python.org>
2. Python Software Foundation — <https://www.python.org>
3. Ubuntu Official Documentation
4. GNU Project Documentation
5. GitHub Open Source Repositories

